

```

1 def can_segment(s, dictionary):
2     dp[0] = True
3
4     for i in range(1, n + 1):
5         for j in range(i):
6             if dp[j] and s[j:i] in words:
7                 dp[i] = True
8                 break
9
10    return dp[n]
11
12 print(can_segment("leetcode", ["leet", "code"]))
13 print(can_segment("applepenapple", ["apple", "pen"]))
14 print(can_segment("catsanddog", ["cats", "dog", "sand", "and",
15 "cat"]))

```

True
True
False

=== Code Execution Successful ===

```

1 def can_segment(s, dictionary):
8     for i in range(1, n + 1):
9         for j in range(i):
10            if dp[j] and s[j:i] in words:
11                dp[i] = True
12                break
13
14     return dp[n]
15
16
17 slug1 = "leet-code".replace("-", "")
18 slug2 = "catsandog".replace("-", "")
19
20 print(can_segment(slug1, ["leet", "code"]))
21 print(can_segment(slug2, ["cats", "dog", "sand", "and",
    "cat"]))

```

True
False

=== Code Execution Successful ===

```
1 def min_segments(s, dictionary):
7
8     for i in range(1, n + 1):
9         for j in range(i):
10             if dp[j] != float('inf') and s[j:i] in words:
11                 dp[i] = min(dp[i], dp[j] + 1)
12
13     if dp[n] == float('inf'):
14         return None
15
16     return dp[n]
17
18
19 print(min_segments("applepenapple", ["apple", "pen"]))
20 print(min_segments("catsandog", ["cats", "dog", "sand", "and",
    "cat"]))
```

3
None

=== Code Execution Successful ===

```
1 def can_segment(s, dictionary):
4
5     dp = [False] * (n + 1)
6     dp[0] = True
7
8     for i in range(1, n + 1):
9         for j in range(i):
10             if dp[j] and s[j:i] in words:
11                 dp[i] = True
12                 break
13
14     return dp[n]
15
16
17 print(can_segment("leetcode", ["leet", "code"]))
18 print(can_segment("applepenapple", ["apple", "pen"]))
```

True

True

=== Code Execution Successful ===

```

1 def suggest_spacing(s, dictionary):
19
20     result = []
21     i = n
22
23     while i > 0:
24         j = parent[i]
25         result.append(s[j:i])
26         i = j
27
28     return " ".join(result[::-1])
29
30
31 print(suggest_spacing("leetcode", ["leet", "code"]))
32 print(suggest_spacing("catsandog", ["cats", "dog", "sand",
    "and", "cat"]))

```

leet code

None

=== Code Execution Successful ===

```
17 def suggest_spacing(s, dictionary):
32     if not dp[n]:
34
35         result = []
36         i = n
37
38         while i > 0:
39             j = parent[i]
40             result.append(s[j:i])
41             i = j
42
43         return " ".join(result[::-1])
44
45
46 print(can_segment("leetcode", ["leet", "code"]))
47 print(suggest_spacing("leetcode", ["leet", "code"]))
```

True

leet code

=== Code Execution Successful ===

```

3 def all_segmentations(s, dictionary):
7     def solve(start):
13         for end in range(start + 1, len(s) + 1):
22             return result
23
24
25     return solve(0)
26
27
28 result = all_segmentations(
29     "catsanddog",
30     ["cats", "cat", "and", "sand", "dog"]
31 )
32
33 for item in result:
34     print(item)

```

```

cat sand dog
cats and dog

```

=== Code Execution Successful ===

```
1 def can_segment(s, dictionary):
8     for i in range(1, n + 1):
9         for j in range(i):
13
14     return dp[n]
15
16
17 print(can_segment(
18     "catsanddog",
19     ["cats", "dog", "sand", "and", "cat"]
20 ))
21
22 print(can_segment(
23     "catsanddog",
24     ["cats", "cat", "and", "sand", "dog"]
25 ))
```

False
True
=== Code Execution Successful ===


```

3 def can_segment(s, dictionary):
7     def solve(start):
8         if start == len(s):
9             return True
10
11         for end in range(start + 1, len(s) + 1):
12             if s[start:end] in words and solve(end):
13                 return True
14
15         return False
16
17     return solve(0)
18
19
20 print(can_segment("leetcode", ["leet", "code"]))
21 print(can_segment("leetcode", ["leet", "code"]))

```

True

True

=== Code Execution Successful ===

```
1 def can_segment(s, dictionary):
7     for i in range(1, len(s) + 1):
8         for j in range(i):
9             if dp[j] and s[j:i] in words:
10                 dp[i] = True
11                 break
12
13     return dp[len(s)]
14
15
16 print(can_segment("leetcode", ["leet", "code"]))
17
18 print(can_segment(
19     "catsandog",
20     ["cats", "dog", "sand", "and", "cat"]
21 ))
```

True
False
Code Execution Successful